Caching strategiyalari

System Design darsi • Uzbek tilida, noldan

Dars maqsadi

Cache nima ekanini, Cache-Aside, Read-Through, Write-Through, Write-Around, Write-Back va Refresh-Ahead strategiyalarini hamda production’dagi xavflarni tushunish.

1. Cache nima?

Cache — tez-tez kerak bo‘ladigan ma’lumotni tezroq ishlaydigan qatlamda vaqtincha saqlash. Masalan, backend PostgreSQL’dan mahsulotni birinchi marta oladi va Redis’ga yozadi; keyingi so‘rovlar Redis’dan tezroq javob olishi mumkin.

Cache hit — ma’lumot cache’da topildi. Cache miss — topilmadi. TTL — cache ma’lumotining amal qilish muddati. Invalidation — eskirgan qiymatni o‘chirish yoki yangilash.

2. Asosiy strategiyalar

Cache-Aside (Lazy Loading)

Backend avval cache’ni tekshiradi. Miss bo‘lsa database’dan oladi va cache’ga yozadi. Mahsulotlar va kategoriya ro‘yxatlari uchun keng tarqalgan, sodda variant. Birinchi so‘rov sekinroq bo‘lishi va cache eskirishi mumkin.

Read-Through

Backend cache qatlamidan o‘qiydi; cache miss bo‘lsa, cache qatlamining o‘zi database’dan oladi va natijani qaytaradi. O‘qish logikasini markazlashtiradi.

Write-Through

Yozish oqimida cache va database yangilanadi. O‘qishda eskirgan qiymat ko‘rinish ehtimolini kamaytiradi, lekin yozish sekinlashishi mumkin. Alohida tizimlarda ikkala yozuv avtomatik atomik bo‘lmaydi.

Write-Around

Yozish to‘g‘ridan-to‘g‘ri database’ga boradi; cache keyinchalik o‘qilganda to‘ldirilishi mumkin. Kam o‘qiladigan ma’lumotlar cache’ni keraksiz to‘ldirmasligi uchun foydali.

Write-Back (Write-Behind)

Avval cache’ga yoziladi, database keyinroq yangilanadi. Tez yozish imkonini beradi, ammo cache yoki jarayon ishdan chiqsa ma’lumot yo‘qolishi mumkin. Moliyaviy va buyurtma ma’lumotlarida qo‘shimcha ishonchlilik talab qilinadi.

Refresh-Ahead

Ma’lumot muddati tugashidan oldin yangilanadi. Ommabop mahsulotlar kabi tez-tez so‘raladigan ma’lumotlarda javobni tezlashtiradi, lekin kam ishlatiladigan ma’lumotni ortiqcha yangilashi mumkin.

3. Tezkor taqqoslash

Strategiya	O‘qish/yozish xususiyati	Asosiy trade-off
Cache-Aside	Backend cache’ni tekshiradi; miss bo‘lsa DB	Sodda; invalidation kerak
Read-Through	Cache miss bo‘lsa cache DB’dan oladi	O‘qish logikasi markazlashadi
Write-Through	Cache va DB yozish oqimida yangilanadi	Yozish sekinroq bo‘lishi mumkin
Write-Around	DB’ga yoziladi; cache keyin to‘ldiriladi	Keraksiz cache yozuvlari kamayadi
Write-Back	Avval cache, keyin DB	Ma’lumot yo‘qolishi xavfi
Refresh-Ahead	Cache oldindan yangilanadi	Ortiqcha refresh bo‘lishi mumkin

4. Cache invalidation misoli

Agar Redis’da mahsulot narxi 100 000 so‘m bo‘lsa va admin PostgreSQL’da uni 120 000 so‘mga o‘zgartirsa, cache’da eski qiymat qolishi mumkin. Oddiy yondashuv: database muvaffaqiyatli yangilangach cache key’ni o‘chirish; keyingi o‘qish yangi qiymatni cache’ga joylaydi.

```
product.price = new_price
product.save()
cache.delete(f"product:{product.id}")
```

Bu soddalashtirilgan misol. Production’da invalidation xatosi, parallel so‘rovlar va database/cache o‘rtasidagi vaqtinchalik nomuvofiqlik ham hisobga olinadi.

5. Production muammolari

Stale data: TTL, invalidation yoki kerakli joyda manbadan qayta tekshirish. Cache stampede: bir kalit muddati tugaganda ko‘p so‘rov DB’ga tushishi; single-flight, lock, TTL jitter yoki refresh-ahead yordam beradi. Noto‘g‘ri cache key: foydalanuvchi/tenant kontekstini hisobga olish. Cache outage: timeout, fallback va DB’ni ortiqcha yuklamaslik.

6. Online shop’da tanlash

• Mahsulotlar va kategoriyalar: Cache-Aside + TTL/invalidation.
• Ommabop mahsulotlar: TTL yoki Refresh-Ahead.
• Sessiya: xavfsiz key va mos TTL.
• Buyurtma summasi/to‘lov holati: cache’ni yagona ishonchli manba qilmaslik.
• Ombordagi oxirgi dona: faqat cache’ga ishonmaslik; atomik database operatsiyasi yoki ishonchli inventory mexanizmi kerak.

7. Amaliy mashqlar

1) Mahsulot ro‘yxati uchun qaysi strategiyani tanlaysan va nima uchun?
2) Narx o‘zgarganda Redis’dagi eski qiymatni qanday boshqarasan?
3) Cache stampede nima va uni qanday kamaytirish mumkin?
4) Nima uchun ombordagi oxirgi dona mavjudligini faqat cache’dan tekshirish xavfli?

Asosiy xulosa: cache tezlikni oshiradi, lekin izchillik, eskirgan ma’lumot, xatolar va yuklamani ham boshqarish kerak. Strategiya ma’lumotning o‘zgarish tezligi va tizimning ishonchlilik talablariga qarab tanlanadi.